SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – CASE STUDY

Course Code:	DSA0502	**Course Title:**	Query Processing for Data Science
CO Assessed:	CO4	**Assessment:**	Tool 1– CASE STUDY
Weightage:		**Total Marks:**	
CO4 Statement: Explore datasets using analytical techniques such as grouping, correlation detection, joining datasets, and extracting meaningful insights.			
Student Name:	Sania Herbert	**Reg. No.:**	192324062
Section / Batch:	2023/AI&DS	**Date of Submission:**	16.08.26
Faculty In-charge:	**Dr. B.Shyamala Gowri**	**Project Mode:**	Individual Submission

Code of Conduct

I Sania Herbert (192324062) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sania

Question 1 – E-Commerce Sales Analysis

An e-commerce company has **Customer, Product, Order, and Payment datasets**. Perform data exploration, join the datasets, analyze sales by product/category/location, identify correlations and outliers, perform time-based analysis, and create suitable visualizations. Provide **5 key insights and recommendations**.

Datasets Used

Four datasets are used:

Dataset	Important Columns	Purpose
Customer	Customer_ID, Customer_Name, Gender, Age, City	Contains customer information
Product	Product_ID, Product_Name, Category, Price	Contains product information
Order	Order_ID, Customer_ID, Product_ID, Order_Date, Quantity, Discount	Contains order details
Payment	Payment_ID, Order_ID, Payment_Method, Payment_Amount	Contains payment information

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# 1 & 2. DATA IMPORTING AND DATA EXPLORATION
# ------------------------------------------------------------
file_path = r"C:\Users\DELL\Downloads\Question_1_Ecommerce_Sales_Datasets.xlsx"
customers = pd.read_excel(file_path, sheet_name="Customers")
products = pd.read_excel(file_path, sheet_name="Products")
orders = pd.read_excel(file_path, sheet_name="Orders")
payments = pd.read_excel(file_path, sheet_name="Payments")
print("\n--- DATASET INFORMATION ---")
print("Customers:", customers.shape)
print("Products:", products.shape)
print("Orders:", orders.shape)
print("Payments:", payments.shape)
print("\nCustomer Data:")
print(customers.head())
print("\nProduct Data:")
print(products.head())
print("\nOrder Data:")
print(orders.head())
print("\nPayment Data:")
print(payments.head())
```

2. Missing Values:

```
print("\nMissing Values:")
print("Customers:\n", customers.isnull().sum())
print("Products:\n", products.isnull().sum())
print("Orders:\n", orders.isnull().sum())
print("Payments:\n", payments.isnull().sum())
print("\nDuplicate Records:")
print("Customers:", customers.duplicated().sum())
print("Products:", products.duplicated().sum())
print("Orders:", orders.duplicated().sum())
print("Payments:", payments.duplicated().sum())
```

```
# ------------------------------------------------------------
# 3. JOINING MULTIPLE DATASETS
# ------------------------------------------------------------
print("\n--- JOINING DATASETS ---")
sales = pd.merge(
    orders,
    customers,
    on="Customer_ID",
    how="left"
)
sales = pd.merge(
    sales,
    products,
    on="Product_ID",
    how="left"
)
sales = pd.merge(
    sales,
    payments,
    on="Order_ID",
    how="left"
)
# Calculate sales
sales["Gross_Sales"] = (
    sales["Price"] * sales["Quantity"]
)
sales["Discount_Amount"] = (
    sales["Gross_Sales"] *
    sales["Discount"] / 100
```

```
)
sales["Final_Sales"] = (
    sales["Gross_Sales"] -
    sales["Discount_Amount"]
)
print("\nIntegrated Dataset:")
print(sales.head())
print("\nIntegrated Dataset Shape:")
print(sales.shape)
```

```python
# ------------------------------------------------------------
# 4. GROUPING & AGGREGATION
# ------------------------------------------------------------
print("\n--- GROUPING & AGGREGATION ---")
# Product
product_sales = sales.groupby(
    "Product_Name"
)["Final_Sales"].sum().sort_values(
    ascending=False
)
print("\nSales by Product:")
print(product_sales)
# Category
category_sales = sales.groupby(
    "Category"
)["Final_Sales"].sum().sort_values(
    ascending=False
)
print("\nSales by Category:")
print(category_sales)
# Location
location_sales = sales.groupby(
    "City"
)["Final_Sales"].sum().sort_values(
    ascending=False
)
print("\nSales by Location:")
print(location_sales)
```

```python
# 5. DATA SEGMENTATION
# ------------------------------------------------------------
print("\n--- CUSTOMER SEGMENTATION ---")
customer_sales = sales.groupby(
    ["Customer_ID", "Customer_Name"]
)["Final_Sales"].sum().reset_index()
def segment(value):
    if value >= 50000:
        return "High Value"
    elif value >= 20000:
        return "Medium Value"
    else:
        return "Low Value"
customer_sales["Customer_Segment"] = (
    customer_sales["Final_Sales"].apply(segment)
)
print(customer_sales)
print("\nSegment Count:")
print(
    customer_sales["Customer_Segment"].value_counts()
)
```

```python
# ------------------------------------------------------------
# 6. CORRELATION ANALYSIS
# ------------------------------------------------------------
print("\n--- CORRELATION ANALYSIS ---")
correlation = sales[
    [
        "Quantity",
        "Price",
        "Discount",
        "Payment_Amount",
        "Final_Sales"
    ]
].corr()
print(correlation)
plt.figure(figsize=(8, 6))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title("E-Commerce Correlation Heatmap")
plt.tight_layout()
plt.show()
```

```python
# ------------------------------------------------------------
print("\n--- OUTLIER DETECTION ---")
Q1 = sales["Final_Sales"].quantile(0.25)
Q3 = sales["Final_Sales"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = sales[
    (sales["Final_Sales"] < lower) |
    (sales["Final_Sales"] > upper)
]
print("Lower Limit:", lower)
print("Upper Limit:", upper)
print("\nNumber of Outliers:", len(outliers))
print("\nOutlier Records:")
print(
    outliers[
        [
            "Order_ID",
            "Customer_Name",
            "Product_Name",
            "Final_Sales"
        ]
    ]
)
plt.figure(figsize=(8, 5))
sns.boxplot(
    x=sales["Final_Sales"]
)
plt.title("Sales Outlier Detection")
```

```python
# ------------------------------------------------------------
# 8. TIME-BASED ANALYSIS
# ------------------------------------------------------------
print("\n--- TIME-BASED ANALYSIS ---")
sales["Order_Date"] = pd.to_datetime(
    sales["Order_Date"]
)
sales["Month"] = sales[
    "Order_Date"
].dt.to_period("M")
monthly_sales = sales.groupby(
    "Month"
)["Final_Sales"].sum()
print("\nMonthly Sales:")
print(monthly_sales)
plt.figure(figsize=(10, 5))
plt.plot(
    monthly_sales.index.astype(str),
    monthly_sales.values,
    marker="o"
)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```python
# 9. VISUALIZATION
# ------------------------------------------------------------
print("\n--- VISUALIZATION ---")
# Category
plt.figure(figsize=(8, 5))
category_sales.plot(
    kind="bar"
)
plt.title("Sales by Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.tight_layout()
plt.show()
# Location
plt.figure(figsize=(8, 5))
location_sales.plot(
    kind="bar"
)
plt.title("Sales by Location")
plt.xlabel("City")
plt.ylabel("Sales")
plt.tight_layout()
plt.show()
# Product
plt.figure(figsize=(10, 5))
product_sales.plot(
    kind="bar"
)
plt.title("Sales by Product")
```

```
plt.title("Sales by Product")
plt.xlabel("Product")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# Sales Distribution
plt.figure(figsize=(8, 5))
sns.histplot(
    sales["Final_Sales"],
    kde=True
)
plt.title("Sales Distribution")
plt.xlabel("Final Sales")
plt.tight_layout()
plt.show()
```

Five Key Findings

1. Students with higher attendance generally show better academic performance, indicating that regular class participation can support learning.
2. Internal assessment marks are related to examination performance, suggesting that continuous assessment can be a useful indicator of final results.
3. Assignment performance contributes to understanding overall student performance, particularly for students who consistently perform well across assessments.
4. Performance varies between departments, indicating that different departments may require different academic support strategies.
5. A group of students falls into the At-Risk category, requiring early intervention from faculty members.

Recommendations

1. Monitor students with low attendance regularly.
2. Provide additional academic support to At-Risk students.
3. Conduct remedial classes before final examinations.
4. Encourage consistent assignment and internal assessment participation.
5. Analyze department-wise performance regularly to identify areas requiring improvement.

```
--- DATASET INFORMATION ---
Customers: (12, 5)
Products: (8, 4)
Orders: (20, 6)
Payments: (20, 4)
```

```
Customer Data:
  Customer_ID  Customer_Name  Gender  Age       City
0        C001  Sania Herbert  Female   22    Chennai
1        C002    Rahul Kumar    Male   24  Bangalore
2        C003     Priya Nair  Female   21      Kochi
3        C004       Arun Raj    Male   28    Chennai
4        C005   Meena Joseph  Female   26  Hyderabad
```

Product Data:

	Product_ID	Product_Name	Category	Price
0	P001	Laptop	Electronics	65000
1	P002	Smartphone	Electronics	30000
2	P003	Running Shoes	Fashion	2500
3	P004	Headphones	Electronics	3000
4	P005	Backpack	Fashion	1800

Order Data:

	Order_ID	Customer_ID	Product_ID	Order_Date	Quantity	Discount
0	O001	C001	P001	2026-01-05	1	5
1	O002	C002	P002	2026-01-08	2	10
2	O003	C003	P003	2026-01-12	2	5
3	O004	C001	P004	2026-02-02	3	10
4	O005	C004	P001	2026-02-15	1	0

Payment Data:

	Payment_ID	Order_ID	Payment_Method	Payment_Amount
0	PAY001	O001	UPI	61750
1	PAY002	O002	Card	54000
2	PAY003	O003	UPI	4750
3	PAY004	O004	Card	8100
4	PAY005	O005	Net Banking	65000

Missing Values:

Customers:

```
Customer_ID      0
Customer_Name    0
Gender           0
Age              0
City             0
dtype: int64
```

Products:

```
Product_ID      0
Product_Name    0
Category        0
Price           0
dtype: int64
```

```
Orders:
 Order_ID        0
Customer_ID     0
Product_ID      0
Order_Date      0
Quantity        0
Discount        0
```

```
dtype: int64
Payments:
 Payment_ID        0
Order_ID           0
Payment_Method     0
Payment_Amount     0
dtype: int64
```

```
Duplicate Records:
Customers: 0
Products: 0
Orders: 0
Payments: 0
```

```
Integrated Dataset:
  Order_ID Customer_ID Product_ID  Order_Date  Quantity  Discount  ... Payment_ID Payment_Method  Payment_Amount Gross_Sales Discount_Amount Final_Sales
0     O001        C001       P001  2026-01-05         1         5  ...     PAY001            UPI           61750       65000          3250.0     61750.0
1     O002        C002       P002  2026-01-08         2        10  ...     PAY002           Card           54000       60000          6000.0     54000.0
2     O003        C003       P003  2026-01-12         2         5  ...     PAY003            UPI            4750        5000           250.0      4750.0
3     O004        C001       P004  2026-02-02         3        10  ...     PAY004           Card            8100        9000           900.0      8100.0
4     O005        C004       P001  2026-02-15         1         0  ...     PAY005    Net Banking           65000       65000             0.0     65000.0
```

```
[5 rows x 19 columns]

Integrated Dataset Shape:
(20, 19)
```

```
--- GROUPING & AGGREGATION ---

Sales by Product:
Product_Name
Laptop            305500.0
Smartphone        135900.0
Smart Watch        33180.0
Office Chair       25650.0
Running Shoes      23125.0
Headphones         20850.0
Backpack            6480.0
Keyboard            4275.0
Name: Final_Sales, dtype: float64

Sales by Category:
Category
Electronics    499705.0
Fashion         29605.0
Furniture       25650.0
Name: Final_Sales, dtype: float64

Sales by Location:
City
Chennai       213425.0
Bangalore     120280.0
Mumbai        102950.0
Kochi          65875.0
Delhi          28300.0
Hyderabad      24130.0
Name: Final_Sales, dtype: float64
```

```
--- CUSTOMER SEGMENTATION ---
   Customer_ID  Customer_Name  Final_Sales Customer_Segment
0         C001  Sania Herbert      74125.0       High Value
1         C002    Rahul Kumar     113800.0       High Value
2         C003     Priya Nair      58750.0       High Value
3         C004       Arun Raj      82100.0       High Value
4         C005   Meena Joseph      24130.0     Medium Value
5         C006   Vikram Singh      15550.0        Low Value
6         C007   Anita Sharma      89650.0       High Value
7         C008    Karthik Rao       6480.0        Low Value
8         C009    Divya Menon       7125.0        Low Value
9         C010      Rohan Das      57200.0       High Value
10        C011     Neha Patel      13300.0        Low Value
11        C012   Aditya Verma      12750.0        Low Value
```

```
Segment Count:
Customer_Segment
High Value      6
Low Value       5
Medium Value    1
Name: count, dtype: int64
```

```
--- CORRELATION ANALYSIS ---
                Quantity     Price  Discount  Payment_Amount  Final_Sales
Quantity        1.000000 -0.634951  0.558839       -0.580110    -0.543866
Price          -0.634951  1.000000 -0.109364        0.951102     0.949218
Discount        0.558839 -0.109364  1.000000       -0.034012     0.000563
Payment_Amount -0.580110  0.951102 -0.034012        1.000000     0.998081
Final_Sales    -0.543866  0.949218  0.000563        0.998081     1.000000
```

```
--- OUTLIER DETECTION ---
Lower Limit: -62559.375
Upper Limit: 125215.625

Number of Outliers: 0

Outlier Records:
```

```
Outlier Records:
Empty DataFrame
Columns: [Order_ID, Customer_Name, Product_Name, Final_Sales]
Index: []
```

```
--- TIME-BASED ANALYSIS ---

Monthly Sales:
Month
2026-01    120500.0
2026-02     85980.0
2026-03     42930.0
2026-04     77625.0
2026-05     76825.0
2026-06     89350.0
2026-07     61750.0
Freq: M, Name: Final_Sales, dtype: float64
```

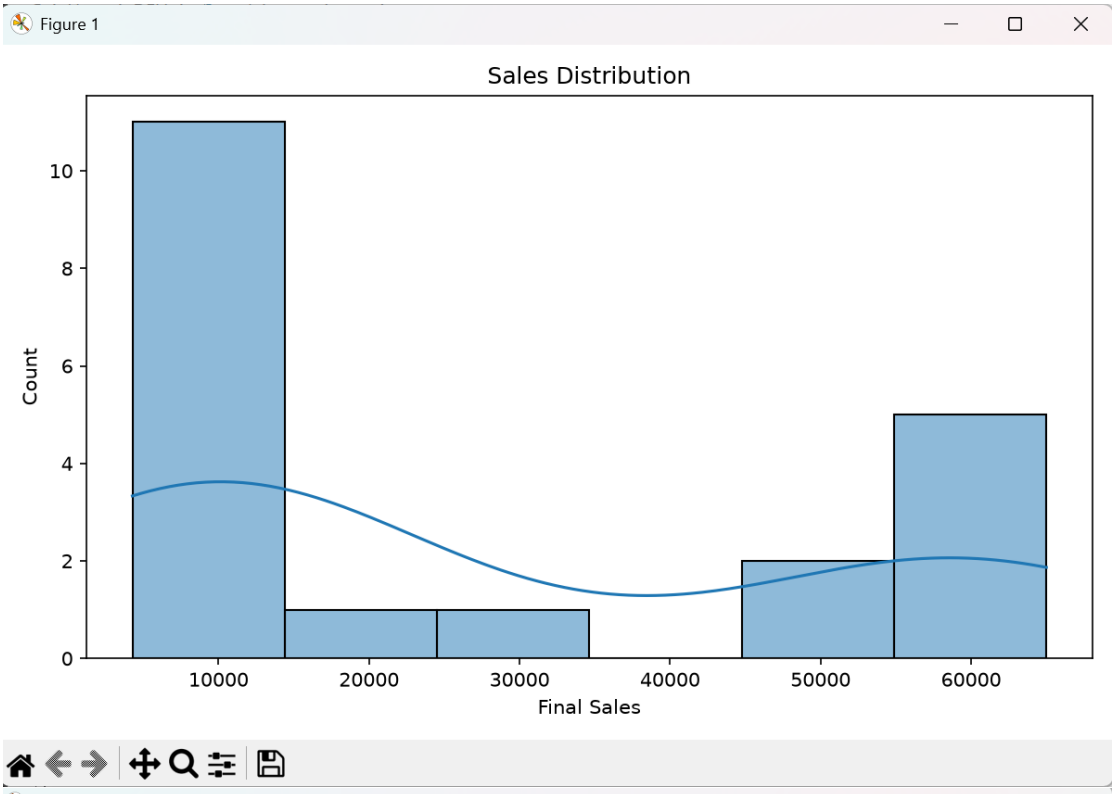


Figure 1
Sales Distribution
Count
Final Sales
0
2
4
6
8
10
10000
20000
30000
40000
50000
60000

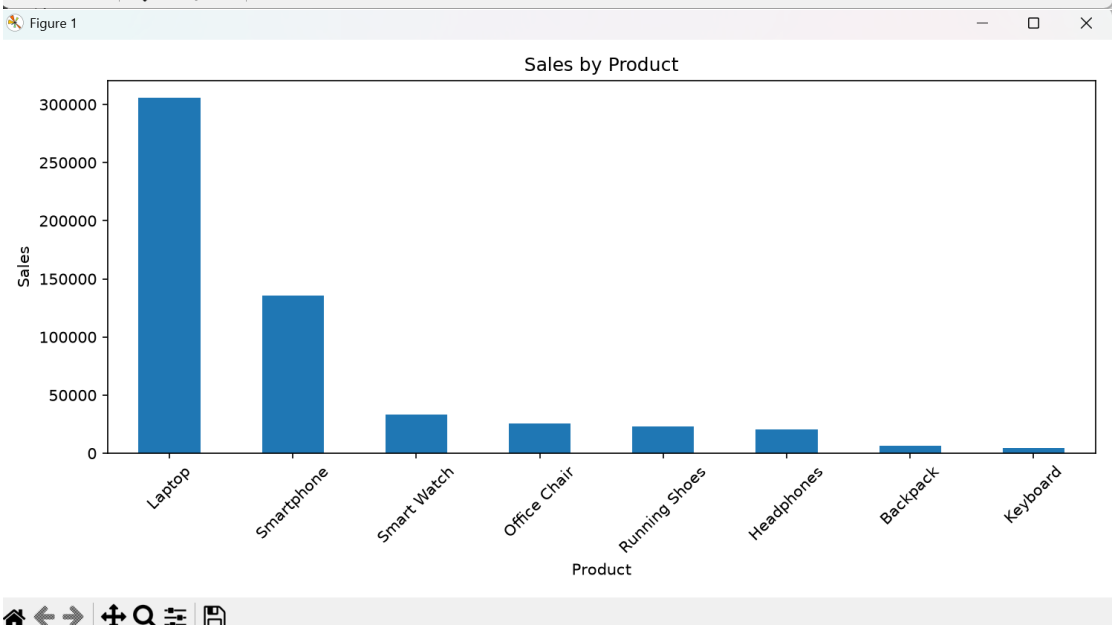


Figure 1
Sales by Product
Sales
Product
0
50000
100000
150000
200000
250000
300000
Laptop
Smartphone
Smart Watch
Office Chair
Running Shoes
Headphones
Backpack
Keyboard

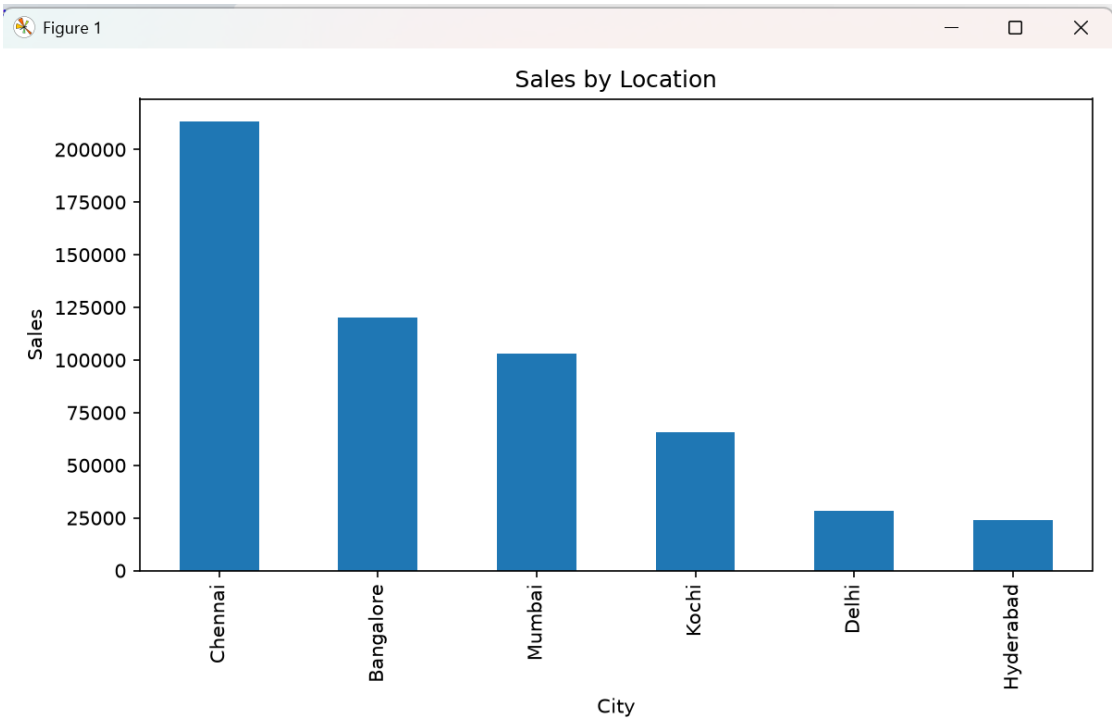


Figure 1
Sales by Location
Sales
200000
175000
150000
125000
100000
75000
50000
25000
0
Chennai
Bangalore
Mumbai
Kochi
Delhi
Hyderabad
City

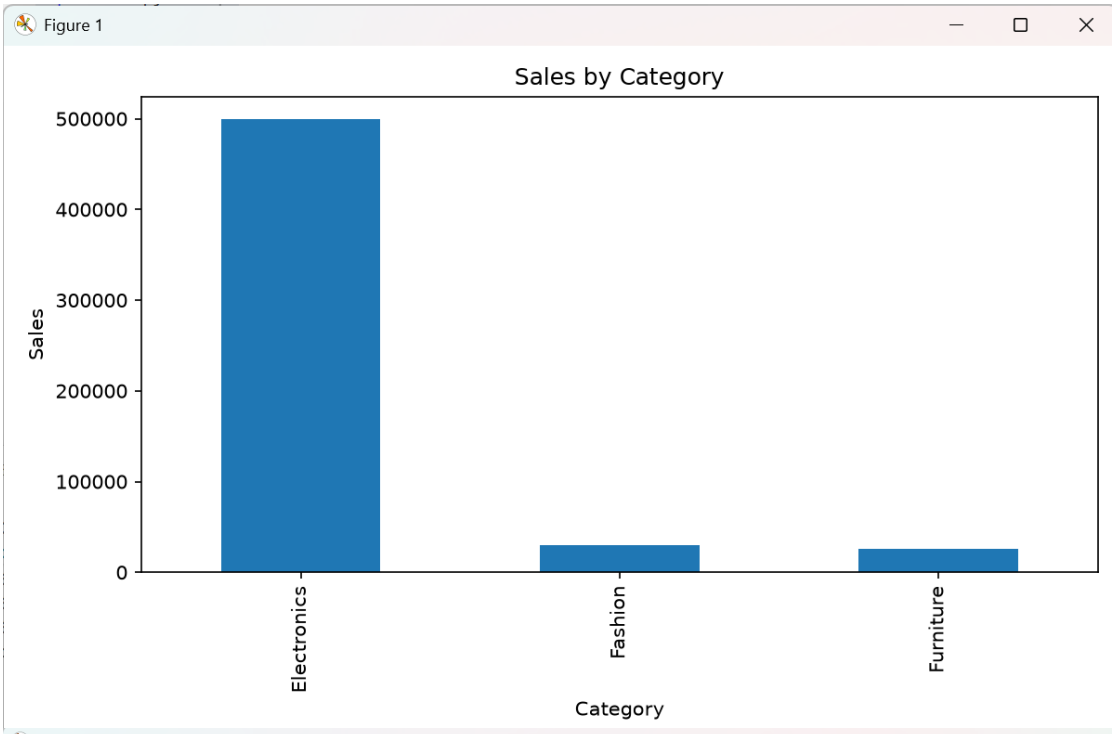


Figure 1
Sales by Category
500000
400000
300000
200000
100000
0
Sales
Electronics
Fashion
Furniture
Category

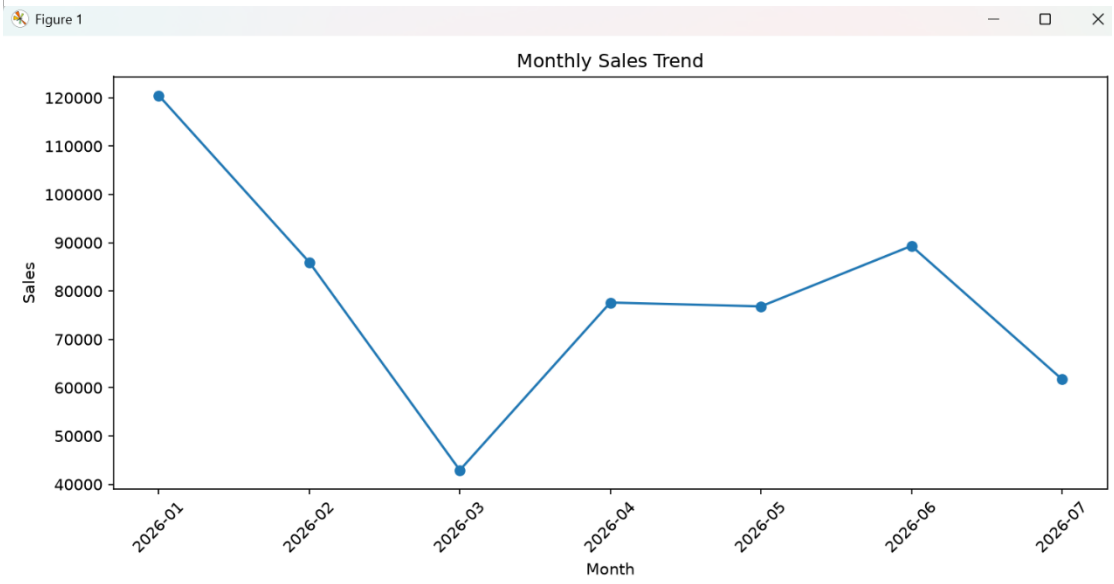
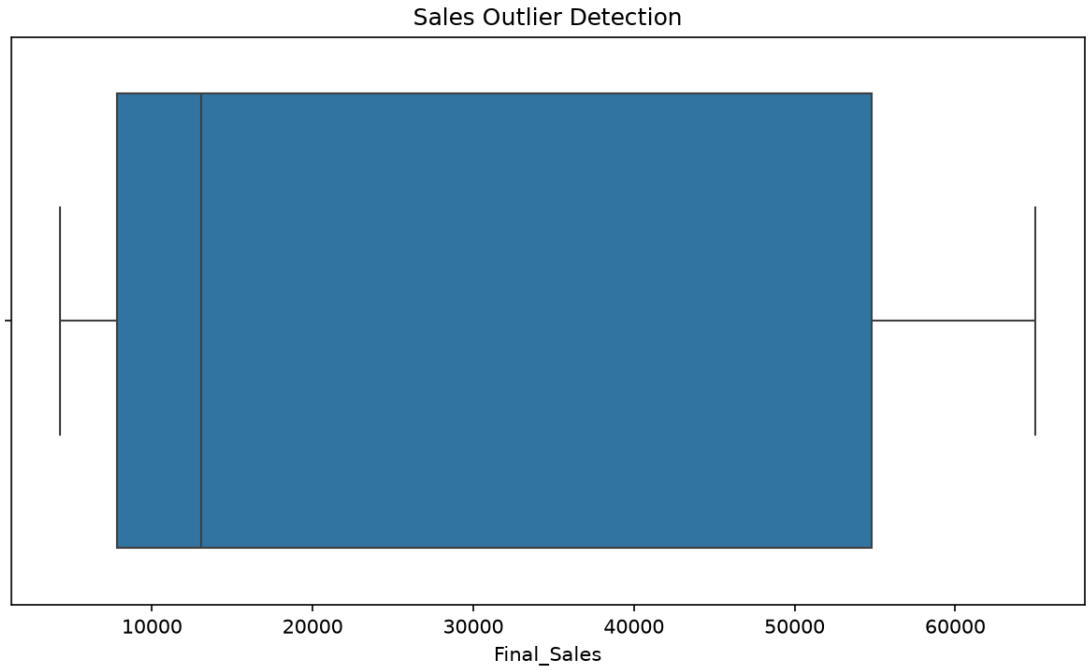
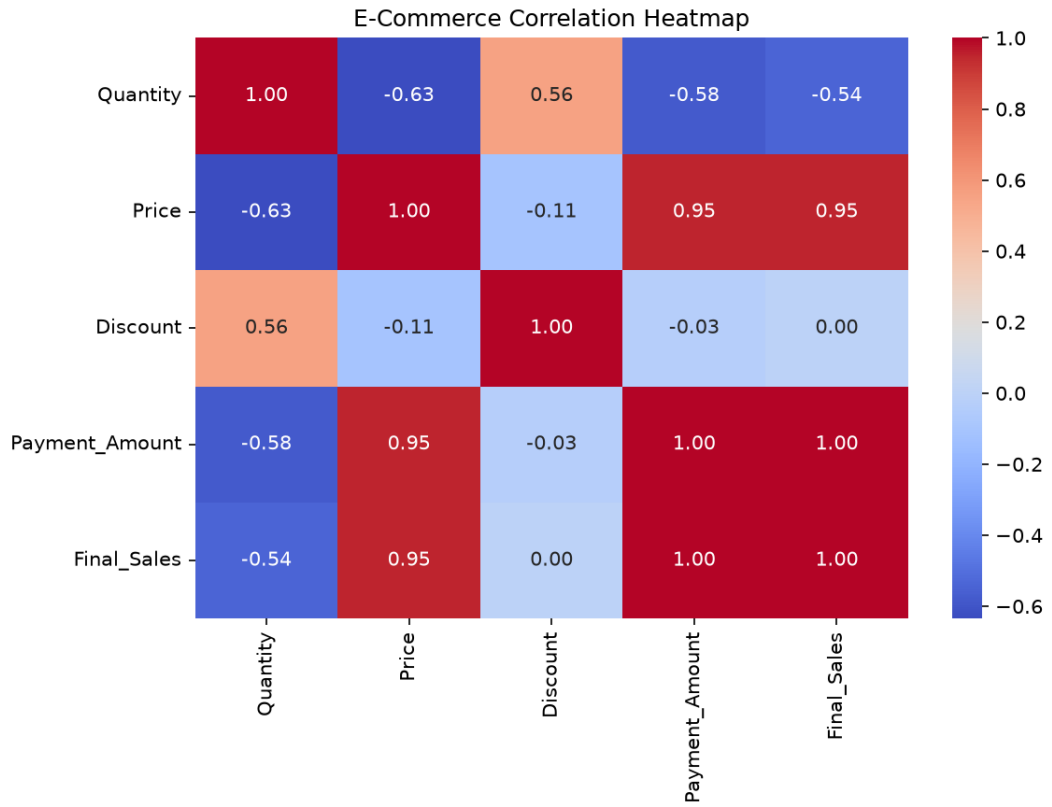


Figure 1
Monthly Sales Trend
120000
110000
100000
90000
80000
70000
60000
50000
40000
Sales
2026-01
2026-02
2026-03
2026-04
2026-05
2026-06
2026-07
Month





Question 2 – Student Academic Performance Analysis

A college has **Student, Attendance, Internal Marks, Assignment, and Examination datasets**. Integrate the datasets and analyze student performance based on department, semester, attendance, and marks. Identify correlations and outliers and visualize performance trends. Provide **5 key findings and recommendations**.

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# ------------------------------------------------------------
# 1 & 2. DATA IMPORTING AND DATA EXPLORATION
# ------------------------------------------------------------
file = r"C:\Users\DELL\Downloads\Question_2_Student_Academic_Performance_Datasets.xlsx"
students = pd.read_excel(
    file,
    sheet_name="Students"
)
attendance = pd.read_excel(
    file,
    sheet_name="Attendance"
)
internal = pd.read_excel(
    file,
    sheet_name="Internal_Marks"
)
assignment = pd.read_excel(
    file,
    sheet_name="Assignment"
)
examination = pd.read_excel(
    file,
    sheet_name="Examination"
)
print("\n--- DATASET INFORMATION ---")
print("Students:", students.shape)
print("Attendance:", attendance.shape)
print("Internal Marks:", internal.shape)
print("Assignment:", assignment.shape)
print("Examination:", examination.shape)
print("\nStudents:")
print(students.head())
print("\nAttendance:")
print(attendance.head())
print("\nInternal Marks:")
print(internal.head())
print("\nAssignment:")
print(assignment.head())
print("\nExamination:")
print(examination.head())
# Missing Values
print("\n--- MISSING VALUES ---")
print("\nStudents:")
print(students.isnull().sum())
print("\nAttendance:")
print(attendance.isnull().sum())
print("\nInternal Marks:")
print(internal.isnull().sum())
print("\nAssignment:")
print(assignment.isnull().sum())
print("\nExamination:")
print(examination.isnull().sum())
# Duplicate Records
print("\n--- DUPLICATE RECORDS ---")
print("Students:",
      students.duplicated().sum())
print("Attendance:",
      attendance.duplicated().sum())
print("Internal Marks:",
      internal.duplicated().sum())
print("Assignment:",
      assignment.duplicated().sum())
print("Examination:",
      examination.duplicated().sum())
```

```python
# ------------------------------------------------------------
# 3. JOINING MULTIPLE DATASETS
# ------------------------------------------------------------
print("\n--- JOINING DATASETS ---")
academic = pd.merge(
    students,
    attendance,
    on="Student_ID",
    how="left"
)
academic = pd.merge(
    academic,
    internal,
    on="Student_ID",
    how="left"
)
academic = pd.merge(
    academic,
    assignment
```

```python
    how="left"
)
academic = pd.merge(
    academic,
    assignment,
    on="Student_ID",
    how="left"
)
academic = pd.merge(
    academic,
    examination,
    on="Student_ID",
    how="left"
)
print("\nIntegrated Academic Dataset:")
print(academic.head())
print("\nIntegrated Dataset Shape:")
print(academic.shape)
```

```python
# ------------------------------------------------------------
# 4. GROUPING & AGGREGATION
# ------------------------------------------------------------
print("\n--- GROUPING & AGGREGATION ---")
# Department performance
department_marks = academic.groupby(
    "Department"
)["Exam_Marks"].mean()
print("\nAverage Exam Marks by Department:")
print(department_marks)
# Department attendance
department_attendance = academic.groupby(
    "Department"
)["Attendance_Percentage"].mean()
print("\nAverage Attendance by Department:")
print(department_attendance)
# Department internal marks
department_internal = academic.groupby(
```

```python
    "Department"
)["Attendance_Percentage"].mean()
print("\nAverage Attendance by Department:")
print(department_attendance)
# Department internal marks
department_internal = academic.groupby(
    "Department"
)["Internal_Marks"].mean()
print("\nAverage Internal Marks by Department:")
print(department_internal)
# Semester performance
semester_marks = academic.groupby(
    "Semester"
)["Exam_Marks"].mean()
print("\nAverage Marks by Semester:")
print(semester_marks)
```

```python
# ------------------------------------------------------------
# 5. DATA SEGMENTATION
# ------------------------------------------------------------
print("\n--- STUDENT PERFORMANCE SEGMENTATION ---")
def performance(mark):
    if mark >= 85:
        return "Excellent"
    elif mark >= 70:
        return "Good"
    elif mark >= 50:
        return "Average"
    else:
        return "At Risk"
academic["Performance_Category"] = (
    academic["Exam_Marks"]
    .apply(performance)
)
print(
    academic[
```

```python
)
print(
    academic[
        [
            "Student_ID",
            "Student_Name",
            "Exam_Marks",
            "Performance_Category"
        ]
    ]
)
print("\nPerformance Category Count:")
print(
    academic[
        "Performance_Category"
    ].value_counts()
)
```

```python
# ------------------------------------------------------------
# 6. CORRELATION ANALYSIS
# ------------------------------------------------------------
print("\n--- CORRELATION ANALYSIS ---")
correlation = academic[
    [
        "Attendance_Percentage",
        "Internal_Marks",
        "Assignment_Marks",
        "Exam_Marks"
    ]
].corr()
print("\nCorrelation Matrix:")
print(correlation)
# Correlation Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(
    correlation,
```

```
]
].corr()
print("\nCorrelation Matrix:")
print(correlation)
# Correlation Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title(
    "Student Academic Correlation Heatmap"
)
plt.tight_layout()
plt.show()
# Attendance vs Exam
plt.figure(figsize=(8, 5))
```

```
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=academic,
    x="Attendance_Percentage",
    y="Exam_Marks"
)
plt.title(
    "Attendance vs Examination Marks"
)
plt.xlabel("Attendance Percentage")
plt.ylabel("Exam Marks")
plt.tight_layout()
plt.show()
```

```
# ------------------------------------------------------------
# 7. OUTLIER DETECTION
# ------------------------------------------------------------
print("\n--- OUTLIER DETECTION ---")
Q1 = academic[
    "Exam_Marks"
].quantile(0.25)
Q3 = academic[
    "Exam_Marks"
].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = academic[
    (academic["Exam_Marks"] < lower) |
    (academic["Exam_Marks"] > upper)
]
print("Lower Limit:", lower)
print("Upper Limit:", upper)
```

```
print("Upper Limit:", upper)
print("\nNumber of Outliers:",
      len(outliers))
print("\nOutlier Students:")
print(
    outliers[
        [
            "Student_ID",
            "Student_Name",
            "Exam_Marks"
        ]
    ]
)
# Boxplot
plt.figure(figsize=(8, 5))
sns.boxplot(
    x=academic["Exam_Marks"]
)
plt.title(
```

```
)
plt.title(
    "Examination Marks Outlier Detection"
)
plt.xlabel("Exam Marks")
plt.tight_layout()
plt.show()
# ------------------------------------------------------------
# 8. TIME-BASED ANALYSIS
# ------------------------------------------------------------
print("\n--- TIME-BASED ANALYSIS ---")
academic["Exam_Date"] = pd.to_datetime(
    academic["Exam_Date"]
)
daily_marks = academic.groupby(
    "Exam_Date"
)["Exam_Marks"].mean()
print("\nAverage Marks by Examination Date:")
print(daily_marks)
```

```
print(daily_marks)
plt.figure(figsize=(10, 5))
plt.plot(
    daily_marks.index,
    daily_marks.values,
    marker="o"
)
plt.title(
    "Examination Performance Trend"
)
plt.xlabel("Examination Date")
plt.ylabel("Average Marks")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
# ------------------------------------------------------------
# 9. VISUALIZATION
# ------------------------------------------------------------
print("\n--- VISUALIZATION ---")
# Department performance
plt.figure(figsize=(8, 5))
department_marks.plot(
    kind="bar"
)
plt.title(
    "Average Exam Marks by Department"
)
plt.xlabel("Department")
plt.ylabel("Average Marks")
plt.tight_layout()
plt.show()
# Department attendance
plt.figure(figsize=(8, 5))
department_attendance.plot(
    kind="bar"
)
plt.title(
    "Average Attendance by Department"
)
plt.xlabel("Department")
plt.ylabel("Attendance Percentage")
plt.tight_layout()
plt.show()
# Performance categories
plt.figure(figsize=(8, 5))
academic[
    "Performance_Category"
].value_counts().plot(
    kind="bar"
)
plt.title(
    "Student Performance Categories"
)
plt.xlabel("Performance Category")
plt.ylabel("Number of Students")
plt.tight_layout()
plt.show()
# Exam marks distribution
plt.figure(figsize=(8, 5))
sns.histplot(
    academic["Exam_Marks"],
    kde=True
)
plt.title(
    "Distribution of Examination Marks"
)
plt.xlabel("Exam Marks")
plt.tight_layout()
plt.show()
# Internal vs Exam
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=academic,
    x="Internal_Marks",
    y="Exam_Marks"
)
plt.title(
    "Internal Marks vs Examination Marks"
)
plt.xlabel("Internal Marks")
plt.ylabel("Exam Marks")
plt.tight_layout()
plt.show()
```

```
--- DATASET INFORMATION ---
Students: (15, 4)
Attendance: (15, 2)
Internal Marks: (15, 2)
Assignment: (15, 2)
Examination: (15, 3)
```

Students:

	Student_ID	Student_Name	Department	Semester
0	S001	Sania Herbert	AI & DS	6
1	S002	Rahul Kumar	CSE	6
2	S003	Priya Nair	AI & DS	6
3	S004	Arun Raj	ECE	6
4	S005	Meena Joseph	CSE	6

Attendance:

	Student_ID	Attendance_Percentage
0	S001	92
1	S002	85
2	S003	96
3	S004	72
4	S005	88

Internal Marks:

	Student_ID	Internal_Marks
0	S001	88
1	S002	76
2	S003	94
3	S004	62
4	S005	81

Assignment:

	Student_ID	Assignment_Marks
0	S001	91
1	S002	78
2	S003	96
3	S004	60
4	S005	84

```
Examination:
  Student_ID  Exam_Marks   Exam_Date
0       S001          89  2026-04-10
1       S002          74  2026-04-10
2       S003          95  2026-04-11
3       S004          58  2026-04-11
4       S005          82  2026-04-12
```

```
--- MISSING VALUES ---

Students:
Student_ID      0
Student_Name    0
Department      0
Semester        0
```

```
Student_Name    0
Department      0
Semester        0
dtype: int64
```

```
Attendance:
Student_ID                0
Attendance_Percentage     0
dtype: int64
```

```
Internal Marks:
Student_ID        0
Internal_Marks    0
dtype: int64
```

```
Assignment:
Student_ID          0
Assignment_Marks    0
dtype: int64
```

```
Examination:
Student_ID    0
Exam_Marks    0
Exam_Date     0
dtype: int64
```

```
--- DUPLICATE RECORDS ---
Students: 0
Attendance: 0
Internal Marks: 0
Assignment: 0
Examination: 0
```

```
Integrated Academic Dataset:
```

	Student_ID	Student_Name	Department	Semester	Attendance_Percentage	Internal_Marks	Assignment_Marks	Exam_Marks	Exam_Date
0	S001	Sania Herbert	AI & DS	6	92	88	91	89	2026-04-10
1	S002	Rahul Kumar	CSE	6	85	76	78	74	2026-04-10
2	S003	Priya Nair	AI & DS	6	96	94	96	95	2026-04-11
3	S004	Arun Raj	ECE	6	72	62	60	58	2026-04-11
4	S005	Meena Joseph	CSE	6	88	81	84	82	2026-04-12

```
Integrated Dataset Shape:
(15, 9)
```

```
Average Exam Marks by Department:
Department
AI & DS    92.166667
CSE        75.400000
ECE        55.500000
Name: Exam_Marks, dtype: float64
```

```
Average Attendance by Department:
Department
AI & DS    93.333333
CSE        84.400000
ECE        71.250000
Name: Attendance_Percentage, dtype: float64
```

```
Average Internal Marks by Department:
Department
AI & DS    90.666667
CSE        76.000000
ECE        59.750000
Name: Internal Marks, dtype: float64
```

```
Average Marks by Semester:
Semester
6    76.8
Name: Exam_Marks, dtype: float64
```

```
--- STUDENT PERFORMANCE SEGMENTATION ---
   Student_ID   Student_Name  Exam_Marks Performance_Category
0        S001  Sania Herbert          89             Excellent
1        S002    Rahul Kumar          74                  Good
2        S003     Priya Nair          95             Excellent
3        S004       Arun Raj          58               Average
4        S005   Meena Joseph          82                  Good
```

ation.py

```
5        S006   Vikram Singh          87             Excellent
6        S007   Anita Sharma          49               At Risk
7        S008    Karthik Rao          72                  Good
8        S009    Divya Menon          94             Excellent
9        S010      Rohan Das          61               Average
10       S011     Neha Patel          80                  Good
11       S012   Aditva Verma          91             Excellent
```

```
Performance Category Count:
Performance_Category
Excellent    6
Good         4
Average      4
At Risk      1
Name: count, dtype: int64
```

```
Correlation Matrix:
                     Attendance_Percentage  Internal_Marks  Assignment_Marks  Exam_Marks
Attendance_Percentage               1.000000        0.995138          0.997140    0.994824
Internal_Marks                      0.995138        1.000000          0.994538    0.998191
Assignment_Marks                    0.997140        0.994538          1.000000    0.994979
Exam_Marks                          0.994824        0.998191          0.994979    1.000000
```

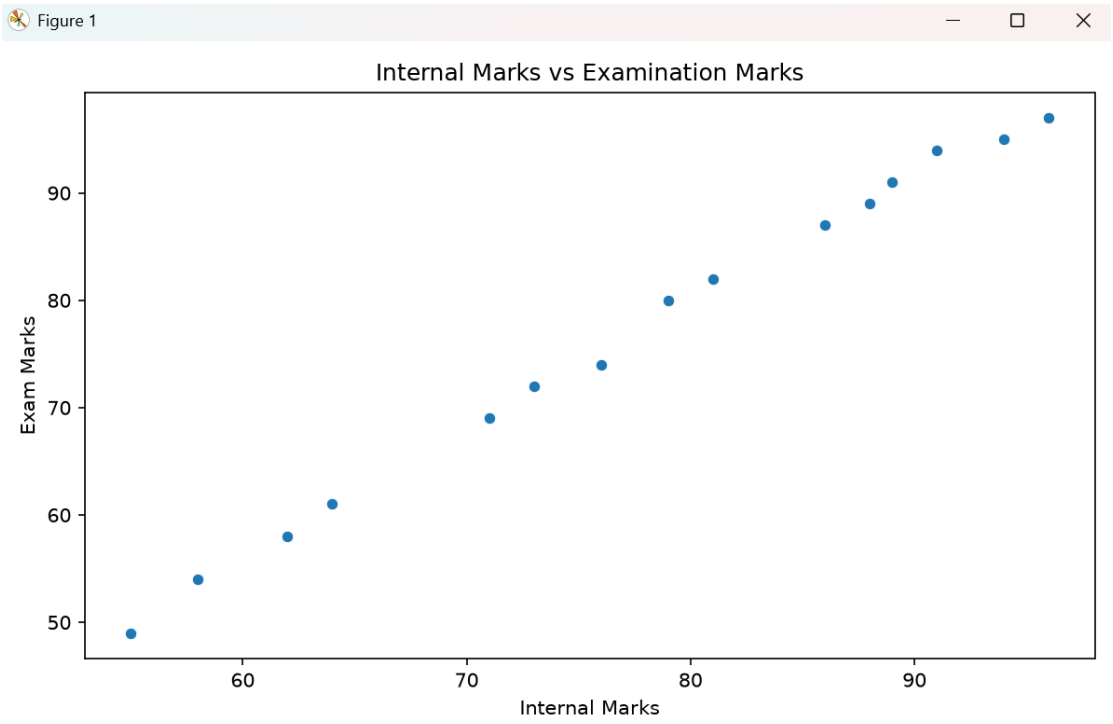
```
--- OUTLIER DETECTION ---
Lower Limit: 27.5
Upper Limit: 127.5

Number of Outliers: 0

Outlier Students:
```

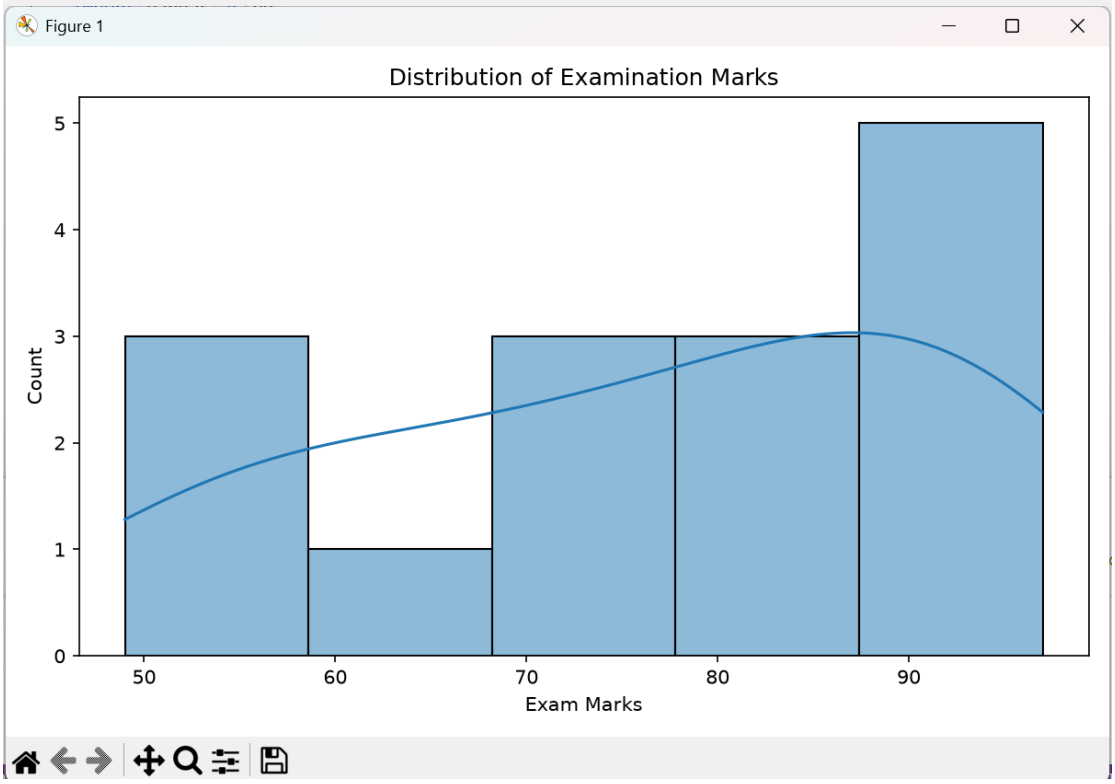
ation.py

```
Outlier Students:
Empty DataFrame
Columns: [Student_ID, Student_Name, Exam_Marks]
Index: []
```

```
Average Marks by Examination Date:
Exam_Date
2026-04-10    81.5
2026-04-11    76.5
2026-04-12    84.5
2026-04-13    60.5
2026-04-14    77.5
```

```
2026-04-14    77.5
2026-04-15    85.5
2026-04-16    61.5
2026-04-17    97.0
Name: Exam_Marks, dtype: float64
```





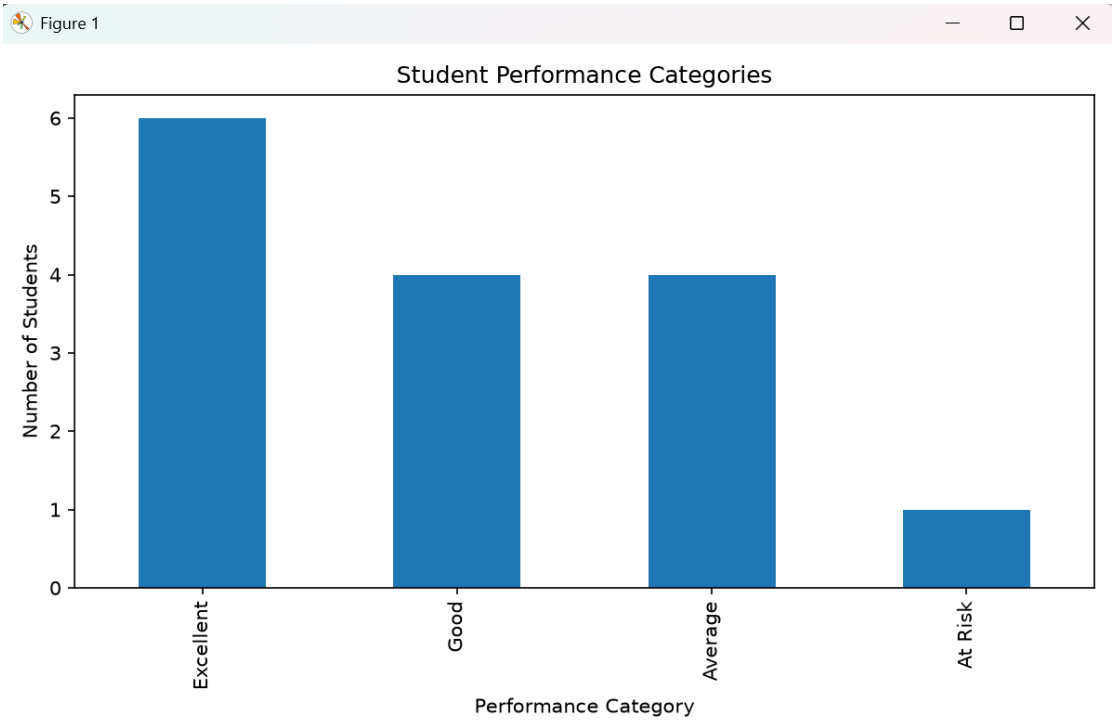
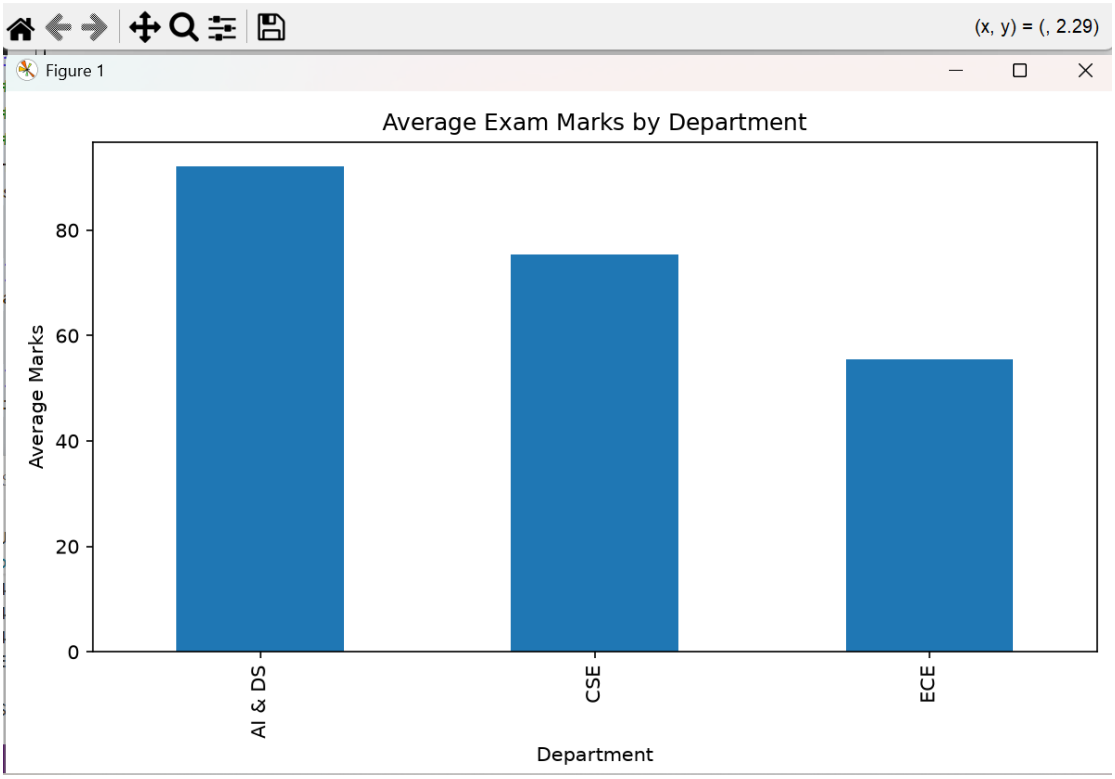


Figure 1
Student Performance Categories
Number of Students
0
1
2
3
4
5
6
Excellent
Good
Average
At Risk
Performance Category


(x, y) = (, 2.29)
Figure 1
Average Exam Marks by Department
Average Marks
0
20
40
60
80
AI & DS
CSE
ECE
Department

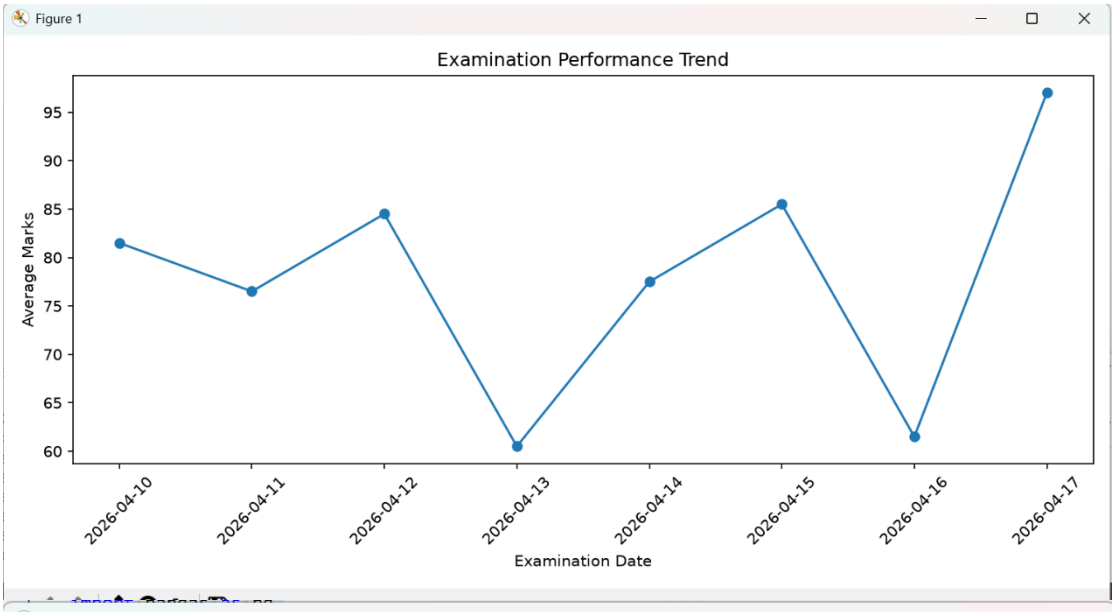


Figure 1
Examination Performance Trend
Average Marks
95
90
85
80
75
70
65
60
2026-04-10
2026-04-11
2026-04-12
2026-04-13
2026-04-14
2026-04-15
2026-04-16
2026-04-17
Examination Date

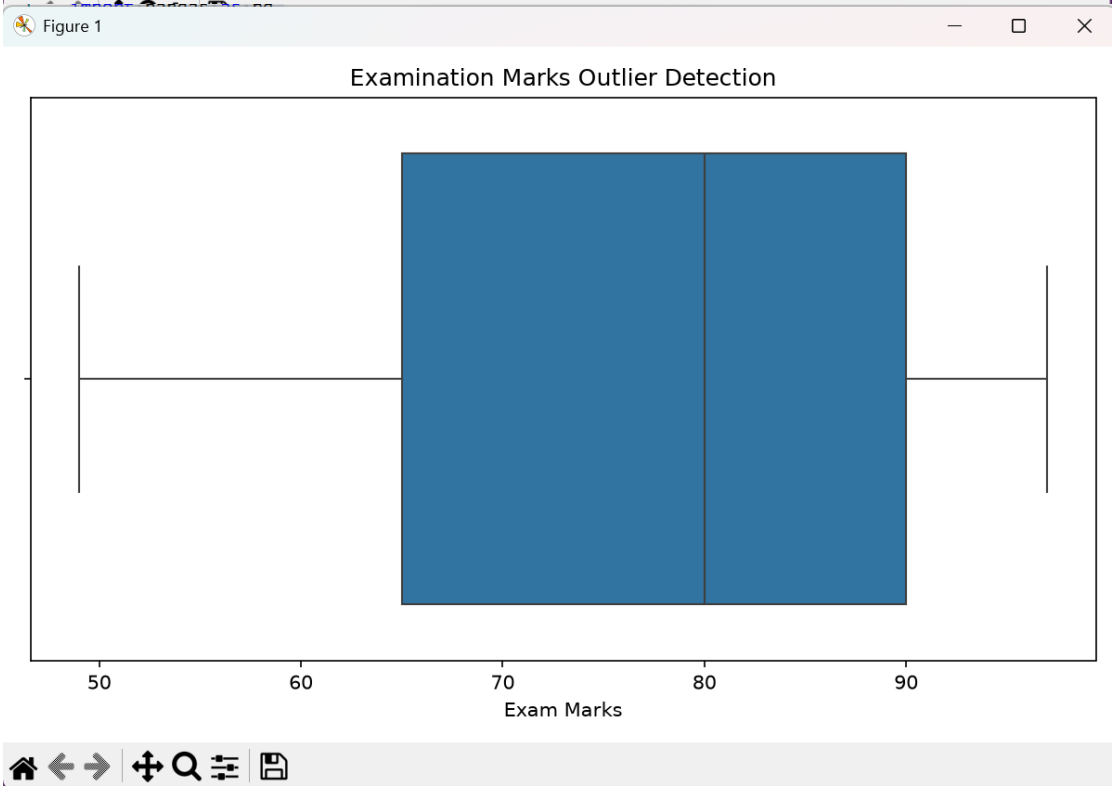
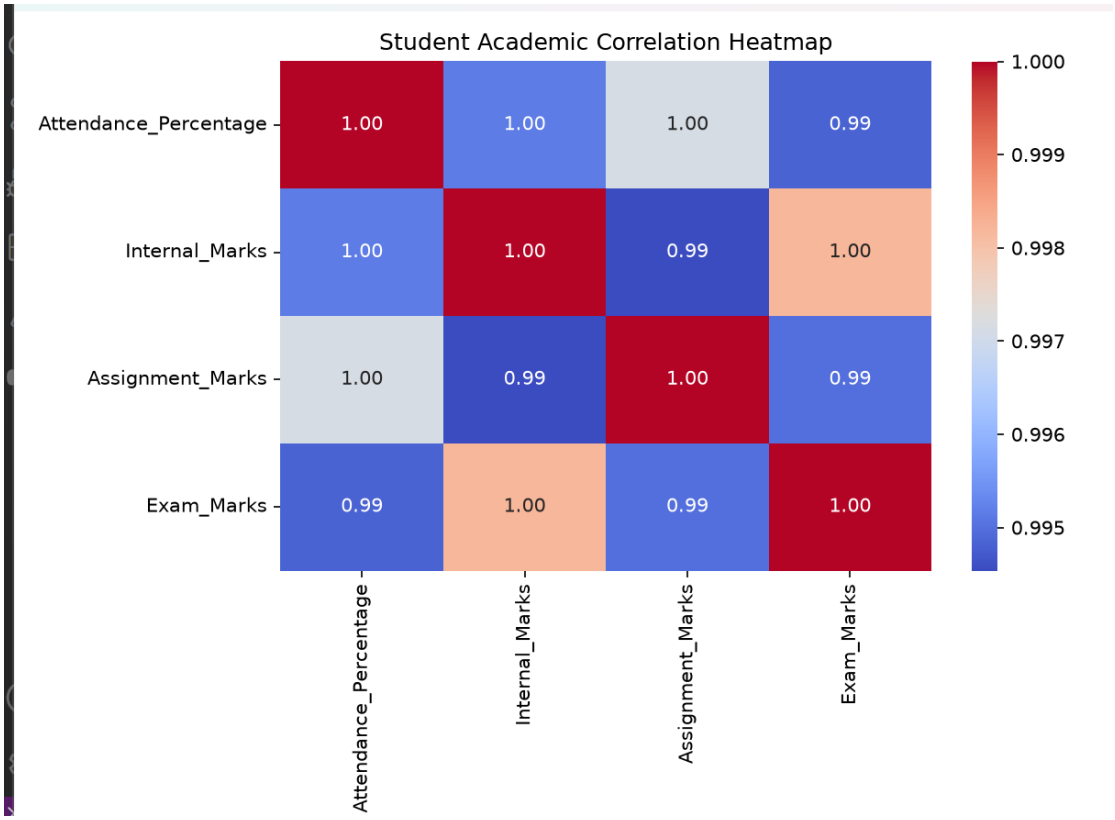


Figure 1
Examination Marks Outlier Detection
50
60
70
80
90
Exam Marks


Student Academic Correlation Heatmap
Attendance_Percentage
Internal_Marks
Assignment_Marks
Exam_Marks
1.00
1.00
1.00
0.99
1.00
1.00
0.99
1.00
1.00
0.99
1.00
0.99
0.99
1.00
0.99
1.00
Attendance_Percentage
Internal_Marks
Assignment_Marks
Exam_Marks
1.000
0.999
0.998
0.997
0.996
0.995